

Figure 4.16 Sending the actual request

4.4.3 Sending the actual request

Once the browser receives a successful preflight response, it sends the actual request to the server, as shown in figure 4.16.

In the blogging app, once the browser receives the preflight response, it sends the HTTP `DELETE` request to delete the blog post. The `DELETE` request can be handled using the same technique you learned in chapter 3: add an Access-Control-Allow-Origin header.

The following listing highlights the code that adds the Access-Control-Allow-Origin header to the response. This code is from chapter 3. You don't need to write any new code—the Access-Control-Allow-Origin header is added to both preflight and actual responses.

Listing 4.9 Adding the Access-Control-Allow-Origin header to all CORS responses

```
var handleCors = function(req, res, next) {
  res.set('Access-Control-Allow-Origin', 'http://localhost:1111');
  if (isPreflight(req)) {
    res.set('Access-Control-Allow-Methods', 'GET, DELETE');
    res.set('Access-Control-Allow-Headers',
            'Timezone-Offset, Sample-Source');
    res.status(204).end();
    return;
  }
  next();
};
```

The Access-Control-Allow-Origin header is added to both preflight and actual CORS requests.

We've spent a lot of time talking about how to successfully respond to a preflight request. But there may be times when you don't want a request to be made. Next, let's turn our attention to rejecting a preflight request.

4.4.4 Rejecting a preflight request

We've explored how to successfully respond to a preflight request. But there may also be times when you want to reject a CORS request. Perhaps your server doesn't support the `DELETE` method at a particular endpoint. How do you tell the browser that the request isn't allowed?